

## Lecture (Cookies + Results + Real API Thinking)

### Learning Outcomes (Tell students)

By the end of this session, you should be able to:

- Explain **stateless HTTP**
- Use **cookies to store user info**
- Understand **HttpContext**
- Use **Results for proper API responses**
- Build a **simple login system**

### ● Part 1 — The Problem (10 min)

**Ask students:**

“If I login, how does the website remember me?”

**Explain:** HTTP is **stateless**

☞ Every request is independent

☞ Server forgets everything

#### **Cookies**

Cookie = small data stored in browser

#### **Flow:**

Server → sends cookie → Browser stores → Browser sends it back

#### **Demo Code:**

```
context.Response.Cookies.Append("username", "Ali");
```

```
var user = context.Request.Cookies["username"];
```

#### **HttpContext**

HttpContext = everything about the current request

#### **Split:**

| Part     | Meaning          |
|----------|------------------|
| Request  | data from client |
| Response | data to client   |

**HttpContext** is Needed for:

- Cookies
- Headers
- Advanced control

#### **Model Binding and HttpContext**

(HttpContext context, LoginRequest request)

**Explain:**

| Item    | Purpose        |
|---------|----------------|
| request | JSON → object  |
| context | cookies + HTTP |

☞ **Golden Rule:**

request = data

context = behavior

#### **Understanding Results in ASP.NET Core**

##### ✦ What is Results?

In ASP.NET Core Minimal API, we use Results to send responses to the client.

Results = how API responds

🔗 It helps us:

- Set the status code
- Return data (JSON)
- Handle success and errors

```
return Results.Ok("Hello");
return Results.BadRequest("Error");
```

| Method         | Meaning      |
|----------------|--------------|
| Ok()           | success      |
| BadRequest()   | client error |
| NotFound()     | missing      |
| Unauthorized() | not allowed  |

#### ♦ Common Types of Results

##### ✓ Success Response

// Return success with data

```
return Results.Ok(new
{
    Status = "Success",
    Message = "Operation completed"
});
```

🔗 Status Code: 200 OK

##### ✗ Error Response

// Return error response

```
return Results.BadRequest(new
{
    Status = "Error",
    Message = "Invalid input"
});
```

🔗 Status Code: 400 Bad Request

##### 🔍 Not Found

// Resource not found

```
return Results.NotFound("User not found");
```

🔗 Status Code: 404 Not Found

##### 🔑 Unauthorized

// User is not allowed

```
return Results.Unauthorized();
```

🔗 Status Code: 401 Unauthorized

#### ♦ Why do we use Results?

Without Results, we cannot clearly control:

- Status code
- Error handling
- API structure

✂ Results makes APIs professional and structured

### Summary

✂ "Model Binding = reading data"

✂ "HttpContext = accessing HTTP details"

✂ "Results = sending response"

### Concept Map (Very Important Summary)

JSON → Model Binding → request object

Cookies → HttpContext → browser memory

Response → Results → client output

"Cookie = small memory stored in browser"

"Server writes → browser keeps → browser sends back automatically"

the response becomes a JavaScript object `.then(res => res.json())`

data is the **response from the backend (API)** after converting it from JSON.

```
.then(data => {  
  document.getElementById("result").innerText = data.message;  
});
```

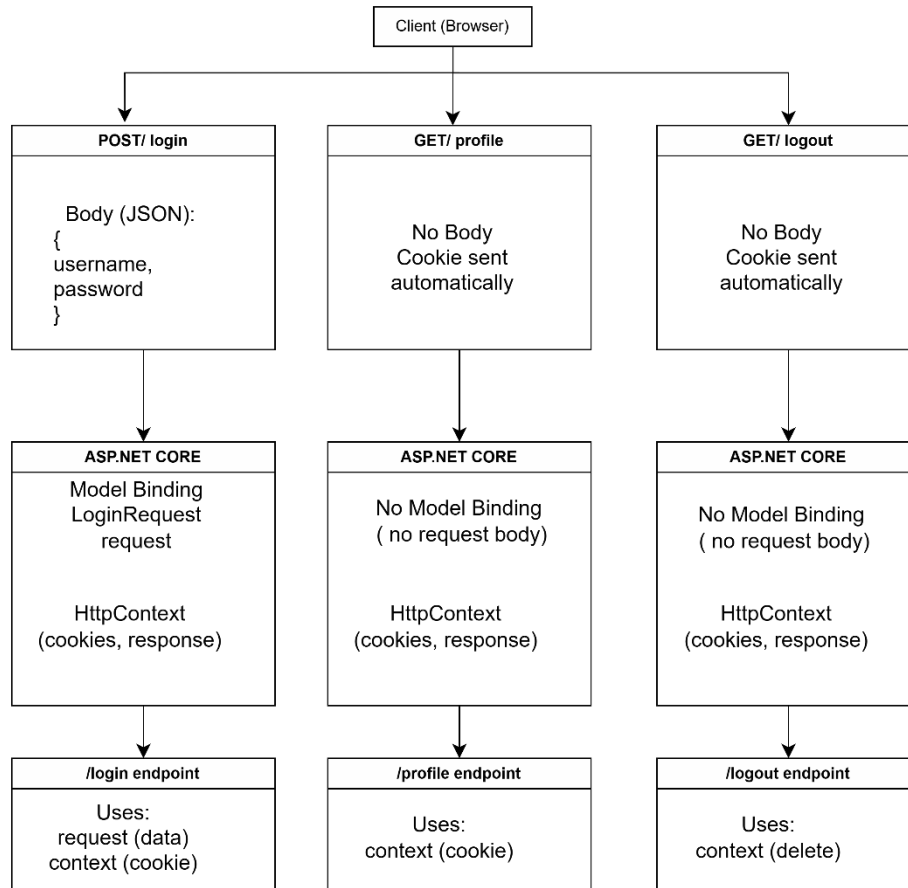
data = whole object

message = one property inside it

HttpContext represents the current HTTP request and response

## Simple Login Full System Flow

Login → store cookie → request profile → read cookie → logout → delete cookie



Students must build:

### 1. Login

- POST request
- Validate username/password
- Store cookie

### 2. Profile

- Read cookie
- Show welcome message

### 3. Logout

- Delete cookie

### 4. Use Results

All endpoints must return:

```
{ "status": "...",  
  "message": "..."  
}
```